

Projeto: Motor de Análise e Logística de Alta Performance

Cenário Corporativo

Vocês foram alocados na equipe de Engenharia de Dados de uma grande varejista de eletrônicos. O sistema legado da empresa utilizava coleções arcaicas como `ArrayList`, o que gerava constantes falhas em tempo de execução por erros de conversão de tipo (`InvalidCastException`) e excessivo consumo de memória devido ao processo de *Boxing*.

A sua missão é reescrever o "Motor de Estoque e Relatórios" utilizando estritamente **C#**. O sistema deve suportar a expansão dinâmica de capacidade, buscas instantâneas sem travamentos, e a geração de relatórios utilizando transformações funcionais (pipelines).

Fase 1: Modelagem do Domínio e Segurança de Identidade

O compilador deve ser o nosso primeiro escudo.

1. Crie uma classe `Produto` contendo as propriedades: `string Sku` (Código Único), `string Nome`, `double Preco` e `int QuantidadeEstoque`.
2. **Identidade e Busca:** Para garantir que a nossa futura coleção de dicionários atinja a performance de $O(1)$ sem falhas, a classe `Produto` deve implementar obrigatoriamente a interface `IEquatable<Produto>`. A igualdade entre dois produtos deve ser definida **exclusivamente** pelo seu `Sku`.
3. **Ordenação Natural:** A classe também deve implementar `IComparable<Produto>`. A ordenação padrão (natural) de uma lista de produtos deve ser sempre da maior quantidade em estoque para a menor.

Fase 2: O Repositório Genérico (Generic Constraints)

Vamos criar uma estrutura reutilizável para armazenar não apenas `Produtos`, mas qualquer entidade futura do sistema.

1. Crie uma classe `Armazem<T>`.
2. Defina **Restrições Genéricas (Constraints)** para garantir que `T` seja obrigatoriamente uma classe (tipo por referência) e que possua um construtor público vazio.
3. Dentro do `Armazem<T>`, instancie uma `List<T>` para armazenar os itens. Crie um método genérico `Adicionar(T item)` que faça a inserção no final da lista.

Fase 3: O Índice de Busca Instantânea

Para evitar o estrangulamento da CPU com varreduras em toda a lista (`List<T>`) durante o fechamento de caixa, implementaremos um índice de busca paralela.

1. Crie uma classe `CatalogoVendas`. Dentro dela, instancie um `Dictionary<string, Produto>`, onde a chave é o `Sku`.
2. Escreva o método `BuscarProdutoParaVenda(string sku)`.
3. **Restrição Crítica:** É **proibido** utilizar a indexação direta (ex: `dicionario[sku]`) para evitar a exceção `KeyNotFoundException` caso o usuário digite um SKU inexistente. Você deve utilizar obrigatoriamente o método `TryGetValue`. Caso o produto não seja encontrado, o método deve retornar um produto nulo ou instanciar um produto com nome "Desconhecido".

Fase 4: O Fantasma da Execução Diferida (LINQ)

O arquiteto júnior da equipe escreveu a seguinte consulta LINQ para notificar o setor de compras sobre produtos que precisam de reposição urgente, mas o relatório está apresentando falhas e dados mutantes:

```
// Código do Arquiteto Júnior
```

```
var produtosEmFalta = listaEstoque.Where(p => p.QuantidadeEstoque < 5); // Linha A
```

```
listaEstoque.Add(new Produto { Sku = "XYZ-99", Nome = "Cabo HDMI", QuantidadeEstoque = 1 }); // Linha B  
DispararEmailParaCompras(produtosEmFalta.Count()); // Linha C
```

Responda de forma discursiva e corrija o código:

1. O novo produto ("Cabo HDMI") inserido na **Linha B** será contabilizado no envio de e-mail na **Linha C**? Justifique detalhadamente com base na teoria de **Avaliação Diferida (Lazy Evaluation)**.
2. Refatore a **Linha A** aplicando o conceito de *Materialização Imediata* para tirar uma 'fotografia' congelada dos dados no momento da consulta, impedindo que mutações posteriores na lista original afetem a variável `produtosEmFalta`.

Fase 5: Abstração e Geração sob Demanda (yield return)

O sistema precisará processar um log de movimentações diárias contendo 5 milhões de registros. Carregar tudo isso na memória causaria o colapso do servidor (`OutOfMemoryException`).

1. Escreva um método chamado `LerLogsMaciços()` que retorne um `IEnumerable<string>`.
2. Utilize a palavra-chave `yield return` para simular a devolução de registros um a um, criando uma **sequência virtual** que pausa a execução e aguarda o consumo, comportando-se como uma "gota d'água" em vez de um "balde cheio".

